

Link Prediction in Apulia Open Data via Graph Autoencoding con Relational Convolutions e ConvE

Anisa Bakiu

Laurea Magistrale in Data Science

Presentazione 2026.1



UniBa

UNIVERSITÀ
DEGLI STUDI
DI BARI
ALDO MORO

Contents

- 1 Motivazione teorica
 - Knowledge Graph Embedding
 - Deep Learning Geometrico
 - Multi-relational GNNs
 - Metriche quantitative di valutazione
- 2 Descrizione del dataset
- 3 Approccio metodologico
- 4 Valutazione sperimentale
- 5 Discussione

Motivazione teorica

Cos'è un grafo?

Matematicamente, un grafo G è definito come una tupla di un insieme di nodi e vertici V e di un insieme di archi e collegamenti.

$$G = (V, E)$$

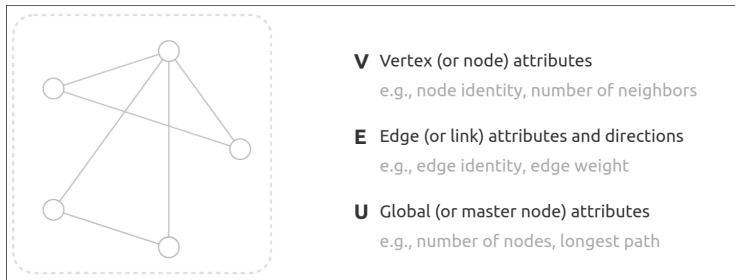


Figure: Struttura del grafo

Task di apprendimento nei grafi

- **A livello di grafo:** apprendimento di una rappresentazione globale per classificazione o regressione dell'intero grafo.
- **A livello di nodo:** previsione dell'identità o del ruolo dei nodi tramite classificazione, regressione o clustering.
- **A livello di arco:** modellazione delle relazioni tra coppie di nodi, includendo classificazione degli archi e link prediction.

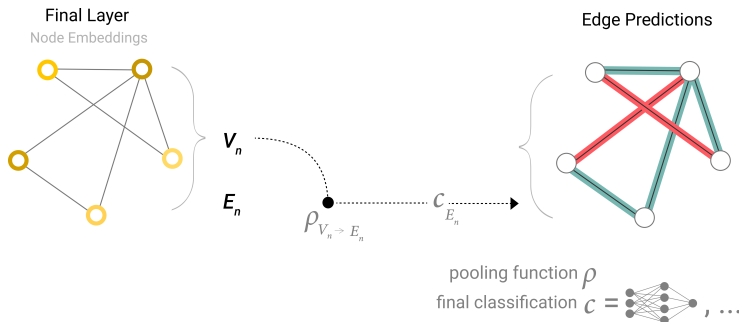


Figure: Stiamo cercando di prevedere se le entità condividono una relazione.

I knowledge graphs

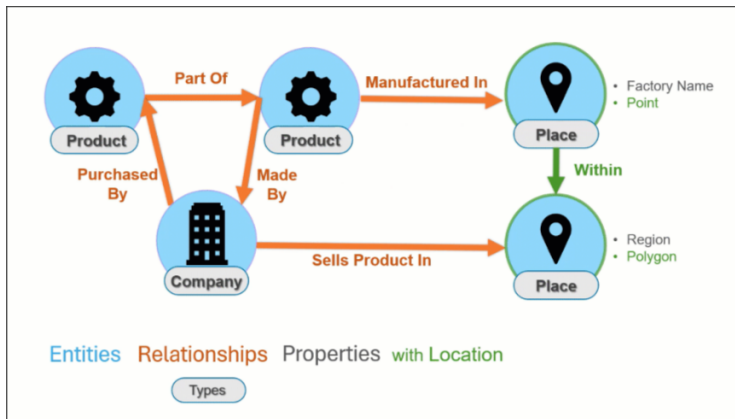


Figure: Esempio illustrativo di un KG

Knowledge Graph Embedding

Il knowledge graph embedding (KGE), consiste nell'apprendere una rappresentazione a bassa dimensione delle entità e delle relazioni di un grafico della conoscenza preservandone il significato semantico.

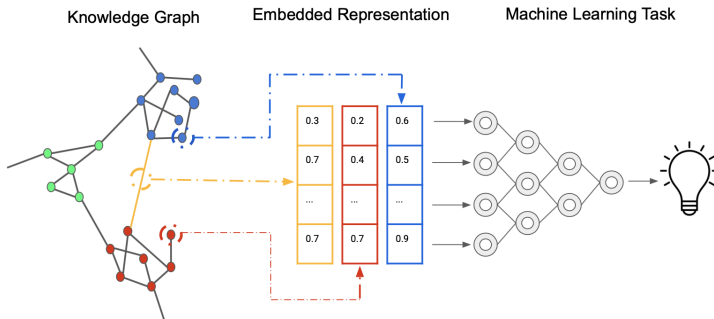


Figure: Esempio illustrativo di un KG

Modelli di decomposizione tensoriale

La decomposizione tensoriale è una famiglia di modelli di embedding che utilizzano una matrice multidimensionale per rappresentare un grafo di conoscenza.

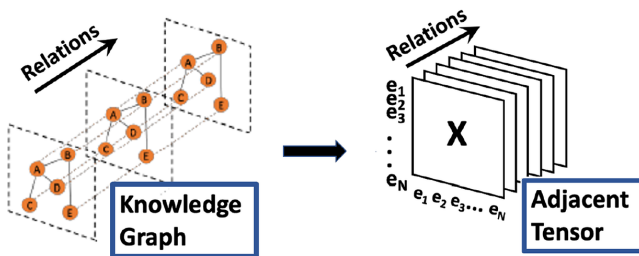


Figure: La decomposizione tensoriale

Modelli bilineari: DistMult

Questo sotto-insieme dei modelli di decomposizione tensoriale, utilizza un'equazione lineare per incorporare la connessione tra le entità tramite una relazione.

DistMult

Poiché la matrice di incorporamento della relazione è una matrice diagonale, la funzione di punteggio non è in grado di distinguere i fatti asimmetrici.

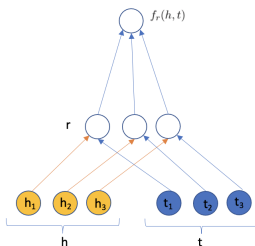


Figure: DistMult model

Modelli di DL convoluzionale: ConvE

Questa famiglia di modelli, impiega uno o più livelli convoluzionali che convolgono i dati di input applicando un filtro a bassa dimensionalità in grado di incorporare strutture complesse con pochi parametri.

ConvE

ConvE utilizza un'embedding di dimensione d per rappresentare le entità e le relazioni. Per calcolare la funzione di score di una tripla:

- 1 prima concatena e unisce gli embedding di "head" della tripla e della relazione "relations" in un singolo dato $[h;r]$,
- 2 e questa matrice viene utilizzata come input per lo strato convoluzionale 2D.

Deep Learning Geometrico

Il deep learning geometrico estende gli approcci tradizionali di deep learning per gestire dati non euclidei, dati che non rientrano in strutture fisse o a griglia regolare come immagini o testo.

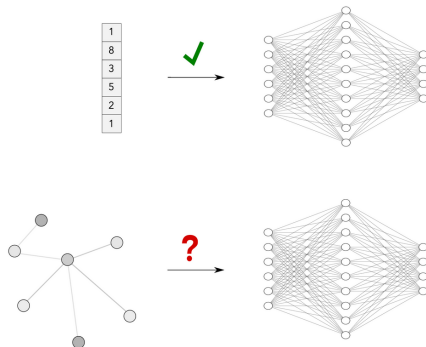


Figure: Le DNN funzionano bene con i vettori. Le cose si complicano con dati non euclidei.

Graph Neural Networks

Le GNN adottano un'architettura graph-in, graph-out, il che significa che questi tipi di modelli accettano un grafo come input, con informazioni nei suoi nodi, archi e contesto globale, e trasformano progressivamente questi embedding, senza modificare la connettività del grafo di input

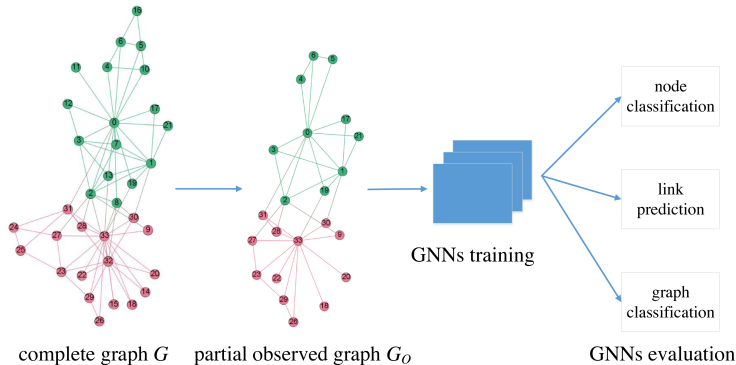


Figure: GNN per le principali task di apprendimento nei grafi

Message passing framework

La caratteristica più distintiva di una GNN è che utilizza una forma di scambio di messaggi neurali in cui i messaggi vettoriali vengono scambiati tra i nodi del grafo e aggiornati utilizzando reti neurali.

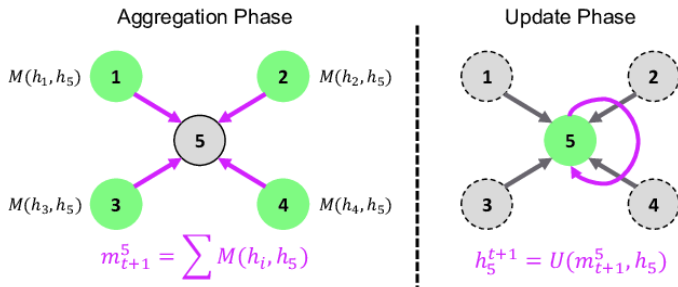


Figure: Aggregation e Update

Graph Convolutional Network

La differenza fondamentale tra le architetture GNN risiede solitamente nel modo in cui gestiscono lo scambio di messaggi.

GCN utilizza operazioni di convoluzione per propagare le informazioni tra i nodi di un grafo.

L'input è un grafo in cui ogni nodo riceve un embedding iniziale basato sui suoi attributi.

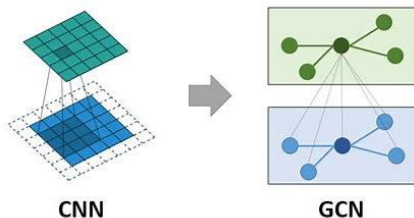


Figure: Analogia con le CNN

Multi-relational GNNs

Finora in questa analisi ho dato per scontato che si tratti di grafi semplici. Tuttavia, esistono numerose applicazioni in cui i grafi in questione sono multirelazionali o comunque eterogenei, ad esempio, i grafi della conoscenza (KG).

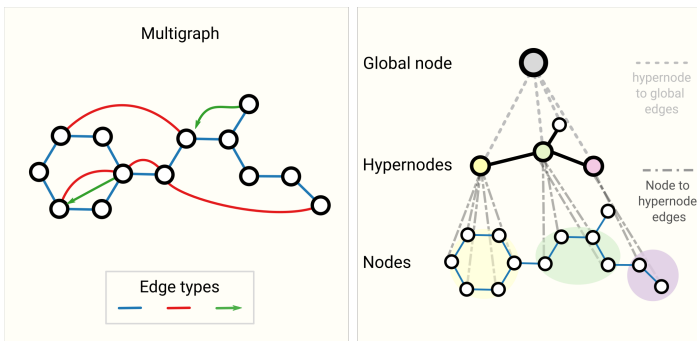


Figure: Schema di grafi più complessi. A sinistra abbiamo un esempio di multigrafo con tre tipi di archi, incluso un arco orientato. A destra abbiamo un grafo gerarchico a tre livelli, i cui nodi di livello intermedio sono ipernodi.

Relational Graph Convolutional Network

Relational Graph Convolution Network (RGCN) è un'implementazione estesa della Graph Convolution Network (GCN). Nella GCN, la rete di grafi non tiene conto delle relazioni tra entità in una base di conoscenza, il che significa che gli archi tra i nodi non vengono considerati. Tuttavia, negli scenari reali, le relazioni (che sono gli archi in un grafo) hanno i propri embedding che devono essere appresi per fornire risultati allo stato dell'arte.

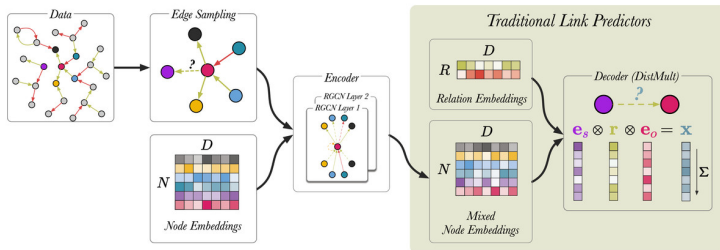


Figure: Una visualizzazione schematica di R-GCN per link prediction

Aggregazione multi-relazionale nelle R-GCN

Per gestire più tipi di relazione, la funzione di aggregazione viene estesa introducendo una matrice di trasformazione distinta per ciascun tipo di arco.

Il messaggio aggregato per un nodo u è definito come:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{\tau \in \mathcal{R}} \sum_{v \in \mathcal{N}_{\tau}(u)} \frac{1}{f_n(u, v)} \mathbf{W}_{\tau} \mathbf{h}_v$$

Con la conseguenza che il modello è in grado di catturare pattern strutturali specifici dei grafi eterogenei.

R-GCN: aggiornamento e regolarizzazione

La rappresentazione nascosta del nodo i al layer $(l + 1)$ è:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\mathbf{W}_0^{(l)} \mathbf{h}_i^{(l)} + \sum_{r \in R} \sum_{j \in N_i^r} \frac{1}{c_{i,r}} \mathbf{W}_r^{(l)} \mathbf{h}_j^{(l)} \right)$$

Però questa rappresentazione presenta un problema: il numero di parametri cresce con il numero di relazioni e questo porta al **rischio di overfitting**.

Per ridurre la complessità e regolarizzare il modello, si utilizza la decomposizione delle matrici di relazione:

$$\mathbf{W}_r^{(l)} = \sum_{b=1}^B a_{rb}^{(l)} \mathbf{V}_b^{(l)}$$

Qui, ogni matrice $\mathbf{W}_r$ è una combinazione lineare di matrici di base $\mathbf{V}_b$ con coefficienti a_{rb} .

Metriche di valutazione

Gli embedding dei modelli possono essere valutati tramite metriche semplici ma efficaci, anche su larga scala. Dato Q come l'insieme di tutte le previsioni classificate di un modello, le principali metriche sono:

- **Hits@K (H@K)**: misura se la previsione corretta è tra le prime K
- **Mean Rank (MR)**: posizione media della previsione corretta
- **Mean Reciprocal Rank (MRR)**: reciproco della posizione, media sui dati

Queste metriche permettono di confrontare facilmente la qualità degli embedding.

Hits@K

Hits@K misura la probabilità di trovare la previsione corretta tra le prime K previsioni di un modello. Solitamente $K = 10$.

$$\text{Hits@K} = \frac{|\{q \in Q : q < K\}|}{|Q|} \in [0, 1]$$

Valori più grandi indicano migliori prestazioni

Rango medio (Mean Rank)

Il rango medio (MR) misura la posizione media delle previsioni corrette tra tutti gli elementi possibili:

$$MR = \frac{1}{|Q|} \sum_{q \in Q} q$$

Permette di capire quanto “in alto” il modello mette le previsioni corrette.

Mean Reciprocal Rank (MRR)

Il Mean Reciprocal Rank ¹ misura quanto velocemente il modello trova la previsione corretta:

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{q} \in [0, 1]$$

Valori più grandi indicano migliori prestazioni.

¹IHR (Inverse Harmonic Rank) è concettualmente equivalente al MRR

Descrizione del dataset

Introduzione al dataset Apulia Travel Open Data

Il dataset utilizzato in questo lavoro "Apulia Travel Open Data", è dedicato alla descrizione di punti di interesse (turistici) nella regione Puglia.

I dati sono rappresentati sotto forma di **Knowledge Graph RDF**, in cui le informazioni sono modellate tramite ontologie semanticamente strutturate e istanze concrete (*ABox*).

Il dataset è organizzato secondo un approccio **ontology-driven**. In particolare:

- 1 Le classi rappresentano concetti semantici.
- 2 Le istanze rappresentano entità reali.
- 3 Le relazioni modellano i legami semantici tra le entità.

Suddivisione dei dati

La Tabella 1 riporta le principali statistiche del dataset dopo la suddivisione in training, validation e test set. Il grafo presenta un numero elevato di entità, con oltre 29 000 nodi nel training set, e più di 65 000 triple.

Table: Statistiche del dataset

	Train	Valid	Test
Entità	29 767	4 966	8 069
Relazioni	25	20	19
Triple (Edges)	65 401	3 847	7 695
Grado medio	2.20	0.77	0.95

Approccio metodologico

Graph Autoencoder con R-GCN + DistMult

L'encoder ha il compito di apprendere una rappresentazione vettoriale continua per ciascuna entità del grafo, sfruttando la struttura relazionale e le informazioni topologiche disponibili.

Il decoder, invece, utilizza tali rappresentazioni per ricostruire gli archi del grafo, assegnando un punteggio alle triple (soggetto, relazione, oggetto).

Table: Sintesi dell'approccio Graph Autoencoder

Componente	Descrizione
Encoder	R-GCN che apprende embedding vettoriali delle entità Sensibile a struttura del grafo e tipi di relazione
Decoder	Funzione di scoring DistMult Valuta le triple candidate (<i>soggetto, relazione, oggetto</i>)
Addestramento	Negative sampling: per ogni tripla osservata vengono generate ω triple negative
Funzione di loss	MarginRankingLoss per ottimizzare il punteggio

Baseline: ConvE

ConvE è l'architettura convoluzionale multistrato più semplice per la previsione dei link, è definita da un singolo strato di convoluzione, uno strato di proiezione sulla dimensione di embedding e uno strato di prodotto interno.

Table: Sintesi dell'approccio ConvE

Aspetto	Descrizione
Architettura	Convoluzione 2D multistrato su embedding Strato di convoluzione, proiezione e prodotto interno
Funzione di scoring	Score alto per triple positive
Training	Triple positive e negative generate tramite negative sampling
Funzione di loss	BCEWithLogitsLoss che combina sigmoid e cross-entropy

Utilizzo di PyKEEN

versione utilizzata PyKEEN 1.11.1

- ❶ le triple del dataset sono state convertite in un formato compatibile con PyKEEN tramite TriplesFactory.
- ❷ Per ogni split del dataset sono state generate le triple costituite da soggetto, relazione e oggetto.
- ❸ Per il training, si generano le mappature tra entità, relazioni e i rispettivi ID numerici.
- ❹ Per gli split di validation e test vengono utilizzate le stesse mappature ottenute dal training.



Figure: PyKEEN library

Ottimizzazione degli iperparametri: R-GCN

Table: Iperparametri principali ottimizzati per R-GCN

Iperparametro	Descrizione
embedding_dim	Dimensione degli embedding delle entità
lr	Passo di apprendimento dell'ottimizzatore Adam

Note: Numero di epoche = 30, batch size = 1024, trials = 5. Ottimizzazione basata su MRR sul set di validazione.

Ottimizzazione degli iperparametri: ConvE

Table: Iperparametri principali ottimizzati per ConvE

Iperparametro	Descrizione
embedding_dim	Dimensione degli embedding delle entità
input_dropout	Dropout sull'input
feature_map_dropout	Dropout sulle feature map
output_dropout	Dropout sull'output
lr	Passo di apprendimento dell'ottimizzatore Adam

Note: Numero di epoche = 30, batch size = 1024, trials = 5. Ottimizzazione basata su MRR sul set di validazione.

Valutazione sperimentale

Risultati del HPO: R-GCN

Table: Iperparametri ottimali per R-GCN

Iperparametro	Valore ottimale
Embedding dimension	128
Numero di layer	2
Bias	No
Funzione di attivazione	LeakyReLU
Dropout archi	0
Dropout self-loop	0.4
Edge weight normalization	inverse_out_degree
Decomposizione pesi	Block
Loss margin	0.717
Learning rate	0.0056
Negativi per positivo	4

Risultati del HPO: ConvE

Table: Iperparametri ottimali per ConvE

Iperparametro	Valore ottimale
Embedding dimension	128
Output channels	32
Input dropout	0.153
Feature map dropout	0.202
Output dropout	0.120
Learning rate	0.0051
Negativi per positivo	5

Risultati: Hits@K

Table: Metriche Hits@K

Model	Hits@1	Hits@3	Hits@5	Hits@10
R-GCN	0.090	0.148	0.174	0.220
ConvE	0.125	0.170	0.190	0.218
Delta Δ	-0.035	-0.022	-0.016	+0.002

Nota: Hits@K misura la probabilità di trovare la tripla corretta nelle prime K posizioni.

Risultati: Metriche di Rank

Table: Metriche di Rank

Model	Median Rank	AMR	GMR	IHR
R-GCN	133	2727.368	155.451	0.137
ConvE	563	7329.143	360.108	0.160
Delta Δ	-430	-4601.775	-204.657	-0.023

Nota: In PyKEEN, IHR è equivalente a MRR.

Risultati: Tempi di addestramento e valutazione

Table: Tempi (in secondi) dei modelli

Modello	Training	Evaluation	Totale
R-GCN	979.5	51.66	1031.16
ConvE	31	5810.22	5841.22
Delta Δ	+948.5	-5758.56	-4810.06

Nota: R-GCN ha tempi di training più lunghi, ma tempi di valutazione molto più brevi rispetto a ConvE.

Discussione

Interpretazione dei risultati

Table: Sintesi delle performance dei modelli

Aspetto	Modello
Recupero dei primi link predetti (Hits@1-5)	ConvE
Recupero di link in una lista più ampia (Hits@10)	R-GCN
Precisione complessiva nella classifica (Median Rank, AMR, GMR)	R-GCN
Capacità di catturare pattern locali ricorrenti	ConvE
Cattura di relazioni complesse e indirette tramite topologia del grafo	R-GCN
Performance su link più probabili	ConvE
Robustezza degli embedding e generalizzazione	R-GCN

Considerazioni sulla stabilità dei modelli

- ConvE: training molto rapido (31s), ma valutazione molto lenta (5810s) dovuto alla verifica di tutti i link inversi.
- R-GCN: training più lungo (979s), ma valutazione molto veloce (51s) che porta ad un'inferenza stabile e scalabile.
- Complessivamente, R-GCN risulta più efficiente durante l'inferenza su grafi grandi.
- Alla fine si tratta di un Trade-off tra accuratezza nel top-k (ConvE) e qualità globale del ranking, e scalabilità di (R-GCN).

Escursioni

Grazie per l'attenzione!

